do280-4.18: Days remaining **89** | Lab hours used **4/80**

DO280-4.22: Days remaining **89** | Lab hours used **0/80**

Red Hat OpenShift Administration II: Configuring a Production Cluster

Course | Lab Environment

Lab: Expose Non-HTTP/SNI Applications

Expose an application to external access without using an ingress controller.

Configure a user-defined network for workload isolation.

Outcomes

- Expose a non-HTTP application to external access by using the `LoadBalancer` type service.
- Create a primary user-defined network to provide workload isolation in a project.
- Deploy an application on both the default pod network and the user-defined network.

As the `student` user on the `workstation` machine, use the `lab` command to prepare your environment for this exercise, and to ensure that all required resources are available:

```
[student@workstation ~]$ lab start non-http-review
```



Instructions

The `lab` command configures the MetalLB operator to provide a single IP address, `192.168.50.20`, for the load balancer services.

The API URL of your OpenShift cluster is `https://api.lab.example.com:6443`, and the `oc` command is already installed on your `workstation` machine.

Log in to the OpenShift cluster as the `developer` user with `developer` as the password. If you need cluster administration privileges, then log in as the `admin` user with `redhatocp` as the password.

1. As the `developer` user, deploy the `virtual-rtsp` application to the new `non-http-review-rtsp` project, and verify that the `virtual-rtsp` pod is running.

   The application consists of the `~/DO280/labs/non-http-review/virtual-rtsp.yaml` file.

   1.1. Log in to your OpenShift cluster as the `developer` user with `developer` as the password.

   ```
   [student@workstation ~]$ oc login -u developer -p developer \
   https://api.lab.example.com:6443
   Login successful.
   ...output omitted...
   ```

   1.2. Change to the `~/DO280/labs/non-http-review` directory.

   ```
   [student@workstation ~]$ cd ~/DO280/labs/non-http-review
   ```

   1.3. Create the `non-http-review-rtsp` project.

   ```
   [student@workstation non-http-review]$ oc new-project non-http-review-
   rtsp
   Now using project "non-http-review-rtsp" on server ...
   ...output omitted...
   ```

   1.4. Use the `oc create` command to create the `virtual-rtsp` deployment by using the `virtual-rtsp.yaml` file.

   ```
   [student@workstation non-http-review]$ oc create -f virtual-rtsp.yaml
   deployment.apps/virtual-rtsp created
   ```

   1.5. List the deployments and pods. Wait for the `virtual-rtsp` pod to be ready. Press **Ctrl**+**C** to exit the `watch` command.

```
[student@workstation non-http-review]$ watch oc get deployments,pods
Every 2.0s: oc get deployments,pods                    ...

NAME                            READY   UP-TO-DATE   AVAILABLE   AGE
deployment.apps/virtual-rtsp    1/1     1            1           21s

NAME                                    READY   STATUS    RESTARTS   AGE
pod/virtual-rtsp-54d8d6b57d-6jsvm       1/1     Running   0          21s
```

Hide Solution

2. Expose the `virtual-rtsp` deployment by using the `LoadBalancer` service.

2.1. Create a load balancer service for the `virtual-rtsp` deployment.

```
[student@workstation non-http-review]$ oc expose deployment/virtual-r
tsp \
--name virtual-rtsp-loadbalancer --type LoadBalancer
service/virtual-rtsp-loadbalancer exposed
```

2.2. Retrieve the external IP address of the `virtual-rtsp-loadbalancer` service.

```
[student@workstation non-http-review]$ oc get svc/virtual-rtsp-loadba
lancer
NAME                       TYPE          ...  EXTERNAL-IP     PORT(S)
virtual-rtsp-loadbalancer  LoadBalancer  ...  192.168.50.20   8554:325
70/TCP
```

The `virtual-rtsp-loadbalancer` service has `192.168.50.20` as the external IP address.

Hide Solution

3. Access the `virtual-rtsp` application by using the URL in the media player. Run the `vlc rtsp://`*`EXTERNAL-IP`*`:8554/stream` command to play the stream in the media player.

3.1. Open the URL in the media player to confirm that the video stream is working correctly.

`rtsp://192.168.50.20:8554/stream`

```
[student@workstation non-http-review]$ vlc rtsp://192.168.50.20:8554/s
tream
...output omitted...
```

You can safely ignore the error messages in the command output.

> **NOTE**
>
> Because the `vlc` command requires a graphical user interface, you cannot use this command when accessing the `workstation` machine from a remote location, such as by using SSH. You must run this command in a terminal on the `workstation` machine.



Close the media player window after confirming that the video stream works correctly.

Hide Solution

4. Create a `udn-nginx` namespace with a primary UDN that uses the `10.42.0.0/24` IP address range. Add the `developer` user as an administrator to the `udn-nginx` project.

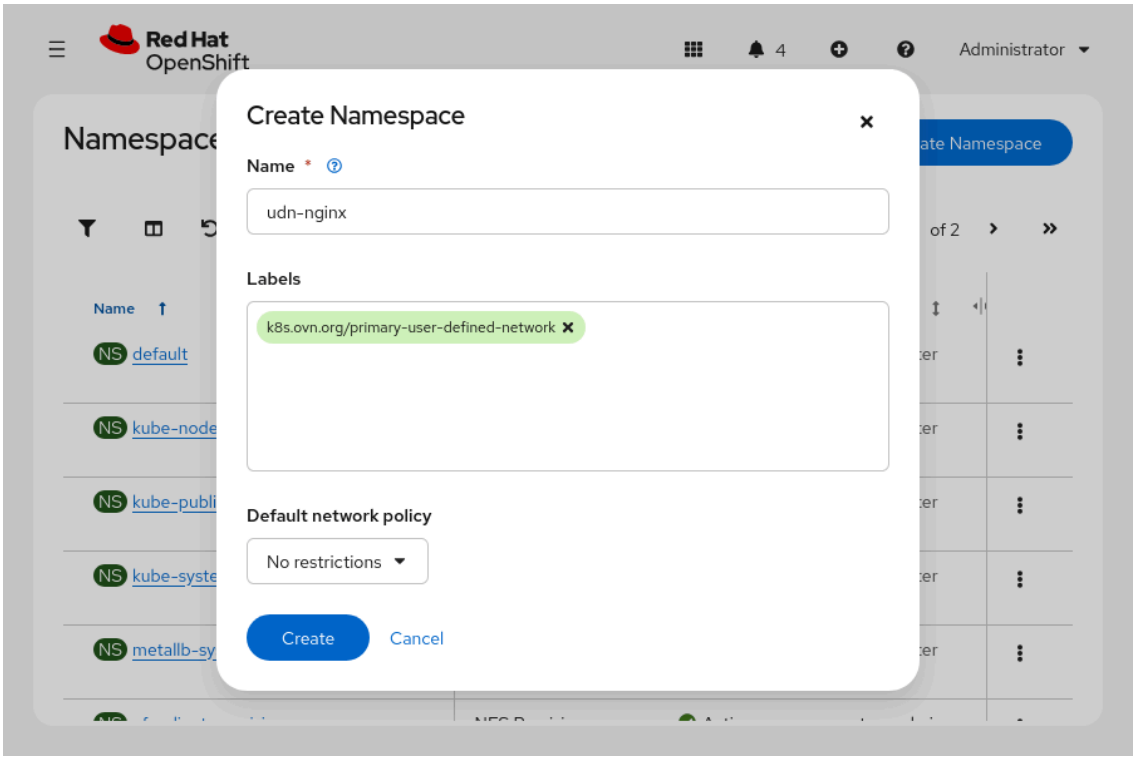
   4.1. Open a web browser and go to the web console URL:

   ```
   https://console-openshift-console.apps.lab.example.com
   ```

4.2. Click **Red Hat Identity Management** and log in as the `admin` user with `redhatocp` as the password.

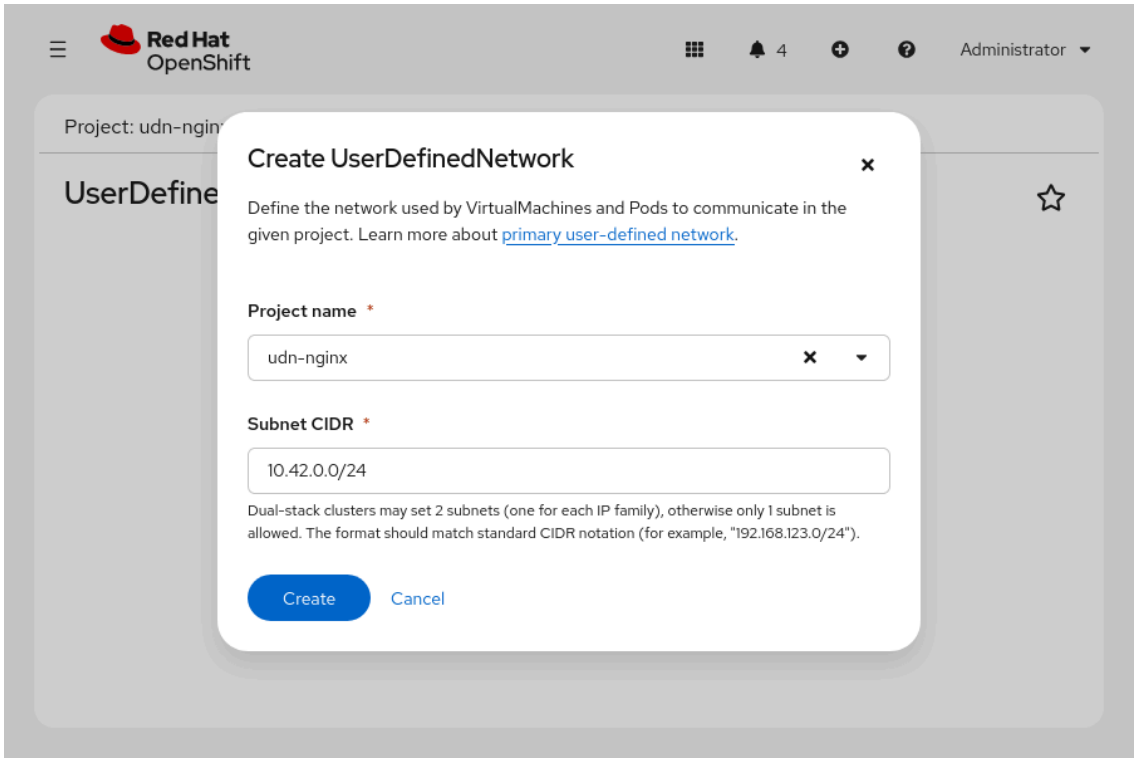
4.3. Go to **Administration** → **Namespaces** and click **Create Namespace**.

4.4. Set the namespace name to `udn-nginx`, add the `k8s.ovn.org/primary-user-defined-network` label, and click **Create**.

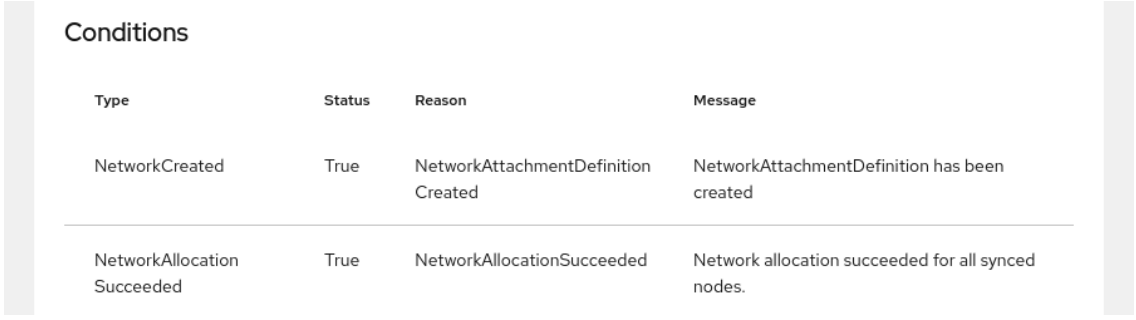


4.5. Go to **Networking** → **UserDefinedNetworks** and click **Create** → **UserDefinedNetwork**.

4.6. Select the **udn-nginx** project, set the IP address range to `10.42.0.0/24`, and click **Create**.



4.7. In the **Conditions** section, verify that both the `NetworkCreated` and `NetworkAllocationSucceeded` conditions are in the `True` state.



4.8. Go to **User Management → RoleBindings** and click **Create binding**.

4.9. Complete the form with the following values:

RoleBinding name	`admin`
Namespace	**udn-nginx**
Role name	**admin**
Subject name	`developer`

Click **Create**.

Hide Solution

5. As the `developer` user, deploy the `nginx` application to the `udn-nginx` project, and verify that the `nginx` pod is running. The application consists of the `~/DO280/labs/non-http-review/nginx.yaml` file.

5.1. From the terminal, change to the `udn-nginx` project.

```
[student@workstation non-http-review]$ oc project udn-nginx
Now using project "udn-nginx" on server "https://api.lab.example.com:
6443".
```

5.2. Use the `oc apply` command to create the `nginx` deployment by using the `nginx.yaml` file.

```
[student@workstation non-http-review]$ oc apply -f nginx.yaml
deployment.apps/nginx created
```

5.3. List the deployments and pods. Wait for the `nginx` pod to be ready. It takes up to one minute for the pod to be ready. The pod name might be different in your environment.

```
[student@workstation non-http-review]$ oc get deployments,pods
NAME                    READY   UP-TO-DATE   AVAILABLE   AGE
deployment.apps/nginx   1/1     1            1           53s

NAME                            READY   STATUS    RESTARTS   AGE
pod/nginx-cdf797bb4-98l9n       1/1     Running   0          53s
```

Hide Solution

6. Scale the `nginx` deployment to two replicas. Verify that the two pods are on the same UDN and that they can communicate with each other by using the `curl` command. The nginx application is configured to listen on port `8080`.

6.1. Scale the `nginx` deployment to two replicas.

```
[student@workstation non-http-review]$ oc scale deployment/nginx --re
plicas 2
deployment.apps/nginx scaled
```

6.2. Verify that the two pods are running. It takes up to one minute for the two pods to be ready.

```
[student@workstation non-http-review]$ oc get pods
NAME                     READY   STATUS    RESTARTS   AGE
nginx-cdf797bb4-98l9n    1/1     Running   0          119m
nginx-cdf797bb4-j4966    1/1     Running   0          11m
```

6.3. List the IP addresses that are assigned to the two `nginx` pods.

```
[student@workstation non-http-review]$ oc get pod/nginx-
cdf797bb4-98l9n \
-o jsonpath='{.metadata.annotations.k8s\.v1\.cni\.cncf\.io/network-st
atus}'
[{
    "name": "ovn-kubernetes",
    "interface": "eth0",
    "ips": [
        "10.8.0.19"
    ],
    "mac": "0a:58:0a:08:00:13",
    "dns": {}
},{
    "name": "ovn-kubernetes",
    "interface": "ovn-udn1",
    "ips": [
        "10.42.0.3"
    ],
    "mac": "0a:58:0a:2a:00:03",
    "default": true,
    "dns": {}
}]
```

```
[student@workstation non-http-review]$ oc get pod/nginx-
cdf797bb4-j4966 \
-o jsonpath='{.metadata.annotations.k8s\.v1\.cni\.cncf\.io/network-st
atus}'
[{
    "name": "ovn-kubernetes",
    "interface": "eth0",
    "ips": [
        "10.8.0.20"
    ],
    "mac": "0a:58:0a:08:00:14",
    "dns": {}
},{
    "name": "ovn-kubernetes",
    "interface": "ovn-udn1",
    "ips": [
        "10.42.0.4"
    ],
    "mac": "0a:58:0a:2a:00:04",
    "default": true,
    "dns": {}
}]
```

The `nginx` pods use the `10.42.0.3` and `10.42.0.4` IP addresses on the UDN. The IP addresses might be different in your environment.

6.4. Run the `curl` command from one pod to verify that it can communicate with the other pod. Use the pod name and IP address from a previous step.

```
[student@workstation non-http-review]$ oc exec -it nginx-
cdf797bb4-98l9n \
-- curl 10.42.0.4:8080
<html>
  <body>
    <h1>Hello, world from nginx!</h1>
  </body>
</html>
```

Hide Solution

7. Deploy another `nginx` application in a new `sdn-nginx` project. Verify that the `nginx` pod on the default pod network cannot communicate with the `nginx` pods on the UDN.

7.1. Create the `sdn-nginx` project.

```
[student@workstation non-http-review]$ oc new-project sdn-nginx
Now using project "sdn-nginx" on server ...
...output omitted...
```

7.2. Use the `oc apply` command to create the `nginx` deployment by using the `nginx.yaml` file.

```
[student@workstation non-http-review]$ oc apply -f nginx.yaml
deployment.apps/nginx created
```

7.3. List the deployments and pods. Wait until the `nginx` pod is running.

```
[student@workstation non-http-review]$ oc get deployments,pods
NAME                    READY   UP-TO-DATE   AVAILABLE   AGE
deployment.apps/nginx   1/1     1            1           18s

NAME                            READY   STATUS    RESTARTS   AGE
pod/nginx-cdf797bb4-kffmq   1/1     Running   0          18s
```

7.4. Run the `curl` command from the pod on the default pod network to verify that the pod cannot communicate with the pods on the UDN. Use the UDN IP address of one of the pods in the `udn-nginx` project from a previous step.

```
[student@workstation non-http-review]$ oc exec -it nginx-
cdf797bb4-kffmq \
-- curl --connect-timeout 5 10.42.0.4:8080
curl: (28) Connection timed out after 5001 milliseconds
command terminated with exit code 28
```

You should get the `Connection timed out` error message.

7.5. Change to the home directory.

```
[student@workstation non-http-review]$ cd
[student@workstation ~]$
```

Hide Solution

Evaluation

As the `student` user on the `workstation` machine, use the `lab` command to grade your work. Correct any reported failures and rerun the command until successful.

```
[student@workstation ~]$ lab grade non-http-review
```

Finish

As the `student` user on the `workstation` machine, use the `lab` command to complete this exercise. This step is important to ensure that resources from previous exercises do not impact upcoming exercises.

```
[student@workstation ~]$ lab finish non-http-review
```

Revision: do280-4.22-ed1-20260827-78c07d2



Privacy Policy (https://www.redhat.com/en/about/privacy-policy?extIdCarryOver=true&sc_cid=701f2000001D8QoAAK)

Red Hat Training Policies (https://www.redhat.com/en/about/red-hat-training-policies)

Terms of Use (https://www.redhat.com/en/about/terms-use)

All policies and guidelines (https://www.redhat.com/en/about/all-policies-guidelines)

Release Notes (https://docs.redhat.com/en/documentation/red_hat_learning_subscription/1-latest/html-single/red_hat_learning_subscription_release_notes/index)

Cookie Preferences and Opt-Out Rights